1 Vacuum cleaner world and agent rationality

Let's consider the vacuum cleaner world. Figure 1 summarizes the characteristics of the environment and of the agent.

Percepts: Location and status, e.g., [A, Dirty]
Actions: Left, Right, Suck, NoOp
Goal: Clean up the rooms + ...
Dirt can be placed statically or dynamically

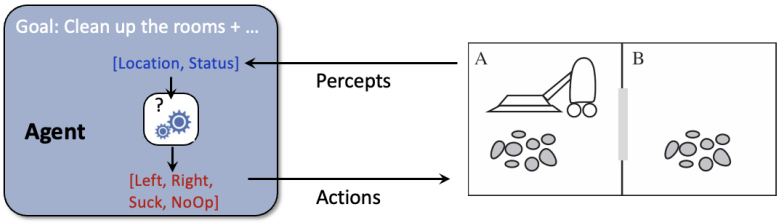


Figure 1: The vacuum cleaner agent and its world environment

The agent function is the one shown below.

```
function Vacuum-Agent ([location, status]) returns an action
   if status == Dirty then return Suck
   else if location == A then return Right
   else if location == B then return Left
```

In principle, the agent keeps acting forever in its environment, aiming, in general, to keep the rooms clean. The scenario can however present different features, that might impact on the *rationality* of the agent given that its behavioral function doesn't change.

For each of the scenario variations below, say whether the agent is rational or not. Briefly justify your answer.

1. Environment is Static, meaning that dirt, if present, is there since the beginning, and no new dirt arrives over time. Goal: Suck all the dirt in the minimum time.

   Solution:
   Dirt is static, all at the beginning, therefore it's rational.

2. Environment is Static. Goal: Maximize collected reward, where +1 is earned each time dirt is cleaned, and -1 is paid each time the robot moves.

   Solution: Not rational, the robot keeps moving, paying -1 at each (useless) move.

3. Environment is Dynamic, meaning that new dirt can arrive at any time. Goal: Maximize collected reward, where +1 is earned each time dirt is cleaned, and -1 is paid each time the robot moves.

Solution:

In general, we shall say that it's not rational. The scenario is complex, it depends on what we know about the process of dirt appearing in the environment. If we don't anything, a rational strategy would be to stay put, waiting for dirt arriving at the current location, and performing some exploratory move to the other room with some low frequency, and maybe adapting the frequency if some pattern of arrival is being spotted. The strategy of keeping moving assumes that dirt keeps arriving regularly with high frequency (i.e., at each time step).

4. Environment is Static, and (all) dirt is placed at the second time step. Location sensor is Unreliable, except for the first location. Goal: Suck all the dirt in the minimum time.

Solution:

Not rational. Let's consider the following instance. The robots knows start location (sensor works for that), it moves (since there's no dirt) and then dirt is placed. Let's assume that robot starts at t=0 in A, and that dirt is only placed in A. At t=1, Robot is in B, but the location sensor says A, therefore it issues Right, with the effect of staying in B, therefore wasting time.

Instead, a rational robot, knowing that location sensor is faulty, would remember the initial known position and the previous action, such that it would know that location is B and therefore would issue Left. In this case, it is clear that it might be necessary to use the entire sequence of percepts and issued actions, in order to issue the optimal decision!

2 Tower of Hanoi

The Tower of Hanoi is a canonical puzzle studying problem solving and formulation. The puzzle starts with n disks of different sizes stacked in order of size on a peg, along with two empty pegs. We can move disks freely between the pegs, but larger disks cannot be stacked on top of smaller ones. The goal is to move all disks to the third peg. Figure 2 shows an example of initial and target configurations for $n = 3$.

We will attempt to formulate Tower of Hanoi as a *search* problem.

1. If we want to represent this puzzle as a search problem, first we need to define the states: what would the states be? Use a python data structure for the purpose. Show the state representation for a few sample states and $n = 2$.

Solution: Store 3 lists tracking the disks stacked on each peg. Enumerate the disks 1 to n.

With only two disks, the states are as follows:

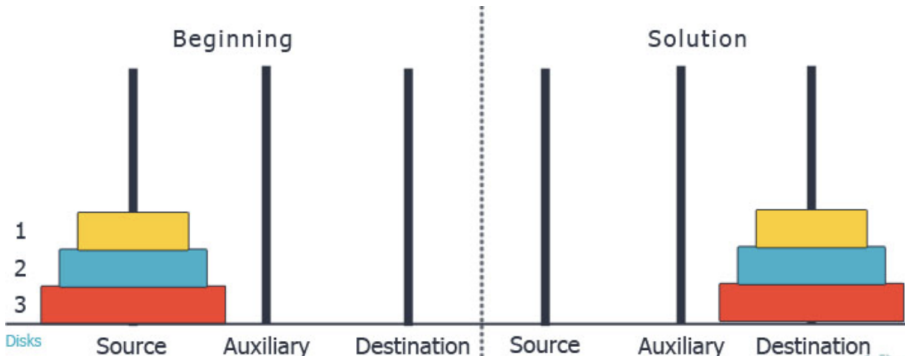


Figure 2: Tower of Hanoi puzzle illustration of initial and target configurations.

- ([1, 2], [], [])
- ([1], [2], [])
- ([1], [], [2])
- ([2], [1], [])
- ([], [1, 2], [])
- ([], [1], [2])
- ([2], [], [1])
- ([], [2], [1])
- ([], [], [1, 2])

For the following questions, assume that you have used your state representation in your answer above.

2. What is the size of the state space in terms of n?

Solution: Assume that a disk number can appear on any of the three lists, independently of where the other numbers are. This means that there are 3^n possible combinations of the three lists. Indeed many of these are not feasible for the problem. But computing the exact feasible number is quite hard.

3. What is the representation of the starting state for generic n?

Solution: ([1, 2, ..., n], [], [])

4. From some given state, what legal actions are there? Describe them.

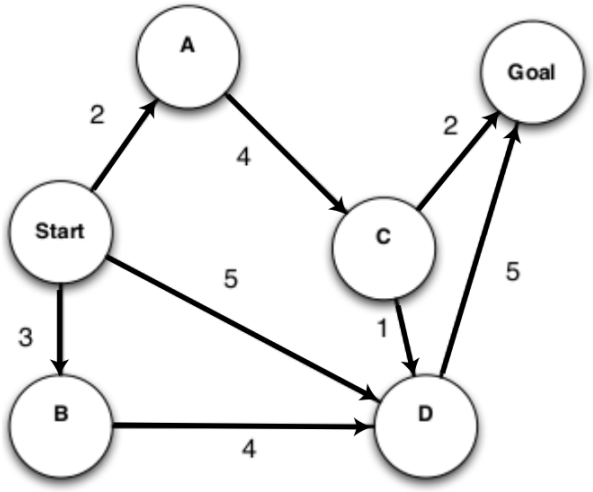
Solution: We can pop the first integer from any list and push it to the front of any other list, given that this integer is less than the current first integer of the target list (or that the target list is empty).

5. What is the goal test (using the state representation)? Remember that this determines whether a given state is a goal state, `goalTest(state)`.

Solution: `goalTest` should check that `state == ([], [], [1, 2, ..., n])`.

3 Warm-up: Uninformed search for a simple problem

The directed state graph shown in the figure represents a search problem with one single goal s tate. The numeric costs of the state transition actions are reported.



1. 4 points Just looking at the problem, can you tell whether BFS will be able to find the minimum cost path to the goal? Justify your answer.

   Solution: Not really. Costs are not uniform and not monotonically non-increasing from start to goal.

2. 3 points What is the average branching factor of the problem?

   Solution: (3+1+1+2+1)/5 = 8/5

3. 3 points DFS tree search comes, usually, with the risk of cycling and no guarantees in terms of optimality. Would these issues be a problem here? In other words, would DFS be a good option to find the path of minimum cost to the goal?

   Solution: There's no problem of cycling here, however, most likely DFS won't find the optimal path, it'll depend on the order to adding the nodes to the frontier.

4. 3 points Now it's time to check what some uninformed search strategies would find once executed. First, without making any path calculation, can you predict which one among BFS, DFS, and UCS for sure will find the minimum cost path to the goal?

   Solution: Yes, UCS will find it.

5. Now, for each one of the algorithms indicated below, work out the returned path from start to goal (use tree search).

   In all cases, assume that nodes are added to the frontier in reverse alphabetical order. E.g., using DFS, when expanding the start node, the successors are included in the frontier in the order D, B, A. Also ties are resolved in the same way, if necessary.

   (a) 4 points DFS:

   Solution: Remember that the convention that we have decided to adopt is that a goal check is only done when a node is expanded, not when it is included in the frontier (indicated with { }).
   {D,B,A} → [A] → {B,D,C} → [C] → {B,D,G,D} → [D] → {B,D,G,G} → [G]
   Path to goal: S, A, C, D, G

   (b) 4 points BFS:

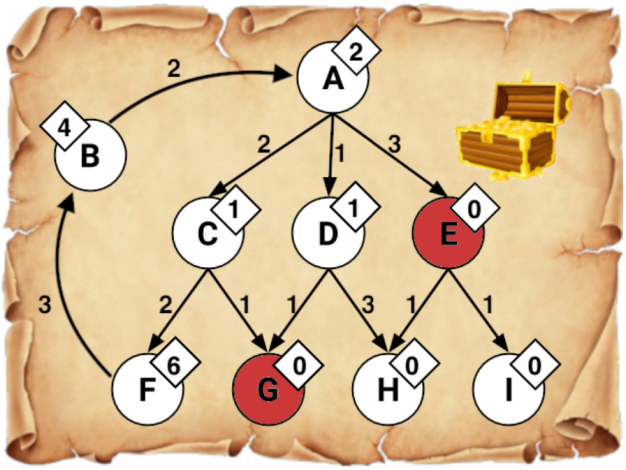
   Solution: {D,B,A} → [D] → {B,A,G} → Goal check ok!
   Path to goal: S, D, G

   (c) 4 points UCS:

   Solution: {D,B,A} → [A] → {D,B,C} → [B] → {D,C,D} → [D] → {C,D,G} → [C] → {D,G,G} → [D] → {G,G,G} → [G] Goal check ok!
   Path to goal: S, A, C, G

4 Search for treasure hunting

We are in the Caribbean and we are trying to find a hidden treasure. We have a treasure map that tells us which paths we can take and how far away the treasure is estimated to be but it's not very accurate. We do know that the map will never overestimate the distance to the treasure. There is a treasure on two different islands and they are equivalent. We only want/need to find one treasure. The map is shown in the figure.



The map shows nine islands scattered around. A is the the island where we are currently with our ship and the red states are the island locations of the treasure. Arrows encode possible navigation actions from each island, and numbers by the arrows represent action costs in terms of time. Note that arrows are directed; for example, $A \rightarrow C$ is a valid action, but $C \rightarrow A$ is not (e.g., because of strong adverse currents).

The numbers shown in diamonds are *heuristic values* that estimate the optimal (minimal) cost to travel from that island to any treasure.

Run each of the following search algorithms with **graph search** and write down the nodes that are added to the *explored set* during the course of the search, as well as the *final path* returned and its cost. Add nodes to the frontier from right to left (e.g., E,D,C when expanding A). When popping off of the frontier, assume that ties are broken alphabetically (e.g., A goes before H).

1. 9 points **Depth-First Search**:
   (a) Explored set:

   Solution: A, C, F, B, G

   (b) Path:

   Solution: A, C, G

   (c) Cost:

   Solution: 3

2. 9 points **Breadth-First Search**:
   (a) Explored set:

Solution: A, E

(b) Path:

Solution: A, E

(c) Cost:

Solution: 3

3. 9 points **Uniform-Cost Search**:

(a) Explored set:

Solution: A, D, C, G

(b) Path:

Solution: A, D, G

(c) Cost:

Solution: 2

4. 9 points **Greedy Search**:

(a) Explored set:

Solution: A, E

(b) Path:

Solution: A, E

(c) Cost:

Solution: 3

5. 9 points **A***:

(a) Explored set:

Solution: A, D, G

(b) Path:

Solution: A, D, G

(c) Cost:

Solution: 2

5 Heuristics and A^*

1. 5 points Depth-first search always expands *at least* as many nodes as A^* search with an admissible heuristic. True or False. Justify your answer (e.g., consider the case where the minimum goal path consists of d nodes).

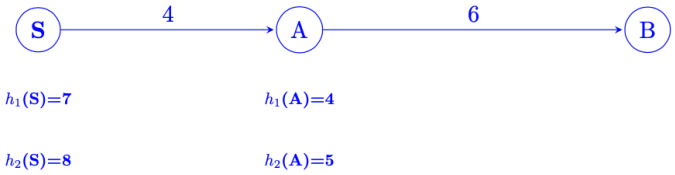
   Solution: False: If the minimum goal path consists of d nodes, a lucky DFS might expand exactly those d nodes to reach the goal. A^* could potentially expand some other nodes before finding that path, more precisely those nodes that would have a lower $f(n)$ value than the final goal's.

2. 5 points Assume that, on a chessboard, for a *single move* a rook can move *any* number of squares in a straight line, either vertically or horizontally, but cannot jump over other pieces. Manhattan distance is an admissible heuristic for the smallest number of moves to move the rook from square A to square B. True or False? Justify your answer.

   Solution: False. A rook can move from one corner to the opposite corner across an NxN board in two moves, although the Manhattan distance from start to finish is is 2(N-1). E.g., 6 steps for a 4x4 grid.

3. 8 points The sum of two (or more) admissible heuristics is still an admissible heuristic (e.g., $h_3(x) = h_1(x) + h_2(x)$). Prove it or disprove it with an example.

   Solution:

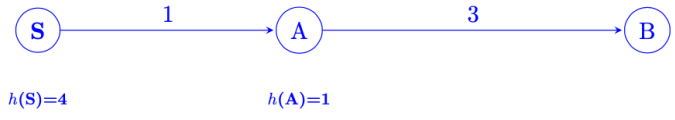
   

   False. Both of these heuristics (h_1 and h_2) are admissible, but if we sum them, we find that $h_3(S) = 15$ and $h_3(A) = 9$. This is not admissible.

   However, notice that taking the maximum of two admissible heuristics will result in an admissible heuristic.

4. 8 points Admissibility of a heuristic for A^* search implies consistency as well. Prove it or disprove it with an example.

   Solution:

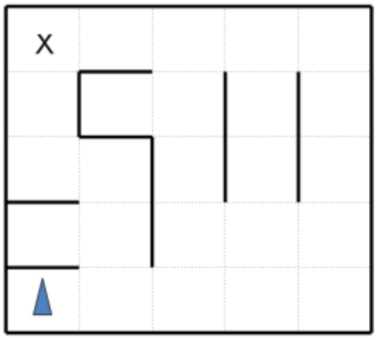
   

   False. Heuristic h is admissible since it never overestimates the true cost to the goal. However, it is not consistent since the step cost (between states S and A) is overestimated by the heuristics. The estimated step cost is $h(S) - h(A) = 4 - 1 = 3$, while the true step cost between S and A is 1.

   Another way of looking at it is $h(S) > cost(S, A) + h(A)$, which violates the triangle inequality.

6 [Bonus] A car agent in a cluttered scenario

Imagine a car-like agent that wishes to park at the x location in the grid environment shown in the figure, featuring walls. The agent is positioned at the bottom-left location.



The agent is directional and at all times faces some direction $d \in \{N, S, E, W\}$. With a single action, the agent can either *move forward at an adjustable velocity* v that can take any value in the discrete set of values $\{0, 1, \ldots, v_{max}\}$, or *turn*.

The turning actions are *left* and *right*, which change the agent's direction by 90 degrees. Turning is only permitted when the velocity is zero (and leaves it at zero).

The moving actions are *fast* and *slow*. Fast increments the velocity by 1 and slow decrements the velocity by 1; in both cases the agent then moves a number of squares equal to its NEW adjusted velocity.

Any action that would result in a collision with a wall crashes the agent and is illegal. Any action that would reduce v below 0 or above the maximum speed v_{max} is also illegal.

The agent's *goal* is to find a plan which parks it (stationary) on the 'x' marked square using as few actions (time steps) as possible.

As an example: if the agent shown were initially stationary, it might first turn to the east using (right), then move one square east using fast, then two more squares east using fast again. The agent will of course have to slow to turn, and so on.

1. 5 points If the grid is M by N, what is the size of the state space? Justify your answer and describe what a state consists of. You should assume that all configurations are reachable from the start state.

Solution:

The size of the state space is $4MN(v_{max} + 1)$.

The state representation is (direction facing, x, y, speed).

2. 6 points Is the Manhattan distance from the agents location to the parking location an admissible heuristic? Yes or No? Justify your answer.

Solution: No, Manhattan distance is not an admissible heuristic since it assumes $v = 1$ constant. Instead, the agent can move at an average speed greater than 1 (e.g., by first speeding up to v_{max} and then slowing down to 0 as it reaches the goal), and so can reach the goal in less time steps than there are squares between it and the goal (i.e., moving at $v = 1$).

A specific example: the target is 6 squares away, and the agent's velocity is already 4. By taking only 4 slow actions, it reaches the goal with a velocity of 0.

3. 8 points State and justify a non-trivial admissible heuristic for this problem.

Solution: There are many possible answers to this question. Here are a few, in order of weakest to strongest:

- The number of turns required for the agent to face the goal.
- Consider a relaxation of the problem where there are no walls, the agent can turn and change speed arbitrarily. In this relaxed problem, the agent would move with v_{max}, and then suddenly stop at the goal, thus taking $d_{manhattan}/v_{max}$ time.
- We can improve the above relaxation by accounting for the deceleration dynamics. In this case the agent will have to slow down to 0 when it is about to reach the goal. Note that this heuristic will always return a greater value than the previous one, but is still not an overestimate of the true cost to reach the goal. This heuristic dominates the previous one.